Prompt - Create a complete MATLAB script demonstrating the real-world application of IIR digital filters using a noisy ECG signal. The goal is to compare three IIR filter designs: 1. Butterworth filter 2. Chebyshev Type I filter 3. Chebyshev Type II filter Requirements: 1. Generate or load a sample ECG-like signal. If an ECG dataset is not available, generate a realistic synthetic ECG signal using MATLAB. 2. Add realistic noise to the ECG signal: - 50 Hz power-line interference - High-frequency Gaussian noise 3. Plot the original clean ECG, noisy ECG, and frequency spectrum of the noisy ECG. 4. Design a low-pass IIR filter using: - Butterworth - Chebyshev Type I - Chebyshev Type II 5. Use suitable sampling frequency and cutoff frequency for an ECG application. Clearly explain the chosen values in comments. 6. Display the magnitude frequency response of all three filters using freqz(). 7. Apply zero-phase filtering using filtfilt() so that the ECG waveform is not phase shifted. 8. Plot the filtered ECG signals from: - Butterworth - Chebyshev Type I - Chebyshev Type II 9. Plot all three filtered signals together for easy comparison. 10. Calculate and display useful performance parameters such as: - Passband ripple - Stopband attenuation - Filter order - Cutoff frequency 11. Clearly explain in MATLAB comments: - What an IIR filter is - Why feedback makes it an IIR filter - Why Butterworth has a flat passband - Why Chebyshev Type I has passband ripple - Why Chebyshev Type II has stopband ripple 12. Add a final comparison figure showing the frequency responses of all three filters. 13. Add a printed summary in the MATLAB Command Window explaining which filter is best for this ECG application and why. 14. Make the code beginner-friendly, well-commented, and executable as a single MATLAB .m file. 15. Use functions from MATLAB Signal Processing Toolbox where appropriate. 16. Make sure there are no undefined variables or missing functions. The final output should be a complete MATLAB program that I can copy, paste, and run directly.

```
% ecg_iir_filter_demo.m
% Demonstration of IIR digital filtering on a synthetic noisy ECG signal.
% Compares Butterworth, Chebyshev Type I, and Chebyshev Type II lowpass
% filters. Uses filtfilt for zero-phase filtering.
%
% Requirements: Signal Processing Toolbox for freqz, cheby1, cheby2.

clearvars; close all; clc;

%% === 1) Simulation parameters and synthetic ECG generation ===
% Sampling frequency and rationale:
% Typical ECG sampling rates range from 250 Hz to 1000 Hz. We choose
% Fs = 360 Hz (commonly used in MIT-BIH arrhythmia database) to capture
% ECG content up to ~100 Hz with adequate resolution.
Fs = 360;               % sampling frequency (Hz)
t_total = 10;           % total seconds
t = (0:1/Fs:t_total-1/Fs)';

% Heart rate and beat template
hr = 60;                % beats per minute
beat_period = 60/hr;    % seconds per beat
num_beats = ceil(t_total/beat_period);

% Create a simple synthetic single-beat ECG template using Gaussians
% Positions (relative to beat start) in seconds for P, Q, R, S, T waves
p_loc = 0.10; q_loc = 0.20; r_loc = 0.22; s_loc = 0.25; t_loc = 0.36;
```

```
% Amplitudes (mV) and widths (s)
p_amp = 0.15; q_amp = -0.12; r_amp = 1.0; s_amp = -0.25; t_amp = 0.35;
p_w = 0.035; q_w = 0.012; r_w = 0.01; s_w = 0.012; t_w = 0.05;

% Build the beat template on a fine grid (one beat length)
tb = 0:1/Fs:beat_period-1/Fs;
beat = p_amp*exp(-((tb-p_loc).^2)/(2*p_w^2)) + ...
       q_amp*exp(-((tb-q_loc).^2)/(2*q_w^2)) + ...
       r_amp*exp(-((tb-r_loc).^2)/(2*r_w^2)) + ...
       s_amp*exp(-((tb-s_loc).^2)/(2*s_w^2)) + ...
       t_amp*exp(-((tb-t_loc).^2)/(2*t_w^2));

% Tile beats to full length and trim
ecg_clean = repmat(beat, 1, num_beats);
ecg_clean = ecg_clean(1:length(t))';

% Add a slow baseline wander (low-frequency respirations)
baseline = 0.05 * sin(2*pi*0.33*t);  % 0.33 Hz respiration
ecg_clean = ecg_clean + baseline;

%% === 2) Add realistic noise ===
% 50 Hz power-line interference (sinusoid)
powerline_freq = 50;                       % Hz
powerline = 0.25 * sin(2*pi*powerline_freq*t); % amplitude ~0.25 mV

% High-frequency Gaussian noise
hf_noise = 0.05 * randn(size(t));         % small Gaussian noise

% Noisy ECG (observation)
ecg_noisy = ecg_clean + powerline + hf_noise;

%% === 3) Plot clean ECG, noisy ECG, and noisy spectrum ===
figure('Name','ECG Signals and Spectrum','NumberTitle','off');
subplot(3,1,1);
plot(t, ecg_clean);
xlim([0 5]); xlabel('Time (s)'); ylabel('mV');
title('Clean Synthetic ECG (first 5 s)');

subplot(3,1,2);
plot(t, ecg_noisy);
xlim([0 5]); xlabel('Time (s)'); ylabel('mV');
title('Noisy ECG (50 Hz power-line + HF Gaussian noise)');

% Frequency spectrum of noisy ECG
Nfft = 2^nextpow2(length(ecg_noisy));
f = Fs*(0:(Nfft/2))/Nfft;
Y = fft(ecg_noisy, Nfft);
Pmag = abs(Y(1:Nfft/2+1));
subplot(3,1,3);
plot(f, 20*log10(Pmag + eps));
```

```
xlim([0 120]); xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
title('Magnitude Spectrum of Noisy ECG (up to 120 Hz)');
grid on;

%% === 4) Design low-pass IIR filters (Butterworth, Cheby I, Cheby II) ===
% ECG content of interest: up to about 40 Hz (QRS has high-frequency content,
% but usually energy above 40-50 Hz is noise). We choose cutoff = 40 Hz.
Fc = 40;                        % desired cutoff frequency (Hz)
Wn = Fc/(Fs/2);                 % normalized cutoff (0..1)

% Filter specifications and choices:
% - Use order 4 for Butterworth (common compromise between roll-off and
stability)
% - For Chebyshev I specify 1 dB passband ripple (rp)
% - For Chebyshev II specify 40 dB stopband attenuation (rs)
butter_order = 4;
cheby1_order = 4; rp = 1;       % passband ripple in dB
cheby2_order = 4; rs = 40;      % stopband attenuation in dB

% Design filters (IIR, direct form via numerator/denominator)
[b_butt, a_butt] = butter(butter_order, Wn, 'low');
[b_cheb1, a_cheb1] = cheby1(cheby1_order, rp, Wn, 'low');
[b_cheb2, a_cheb2] = cheby2(cheby2_order, rs, Wn, 'low');

%% === 5) Display magnitude frequency response of all three filters ===
nfft_freqz = 2048;
[H_butt, fHz] = freqz(b_butt, a_butt, nfft_freqz, Fs);
[H_cheb1, ~] = freqz(b_cheb1, a_cheb1, nfft_freqz, Fs);
[H_cheb2, ~] = freqz(b_cheb2, a_cheb2, nfft_freqz, Fs);

figure('Name','Filter Frequency Responses','NumberTitle','off');
plot(fHz, 20*log10(abs(H_butt)+eps), 'LineWidth', 1.4); hold on;
plot(fHz, 20*log10(abs(H_cheb1)+eps), 'LineWidth', 1.1);
plot(fHz, 20*log10(abs(H_cheb2)+eps), 'LineWidth', 1.1);
xlim([0 120]); ylim([-80 5]);
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
legend('Butterworth (order 4)', sprintf('Cheby I (order 4, rp=%gdB)', rp),
...
       sprintf('Cheby II (order 4, rs=%gdB)', rs), 'Location','southwest');
title('Magnitude Frequency Responses');
grid on;

%% === 6) Apply zero-phase filtering ===
% filtfilt is used to avoid phase distortion (it applies the filter forward
% and backward). This preserves the ECG waveform shape.
ecg_butt = filtfilt(b_butt, a_butt, ecg_noisy);
ecg_cheb1 = filtfilt(b_cheb1, a_cheb1, ecg_noisy);
ecg_cheb2 = filtfilt(b_cheb2, a_cheb2, ecg_noisy);

%% === 7) Plot filtered ECG signals individually and together ===
```

```
xlim([0 120]); ylim([-80 5]);
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
legend('Butterworth','Cheby I','Cheby II','Location','southwest');
title('Comparison of Filter Magnitude Responses');
grid on;

%% === 11) Printed summary recommendation ===
% Simple recommendation logic: for ECG we want minimal passband distortion
% (no ripple) and sufficient attenuation of 50 Hz line noise.
% - Butterworth: smooth passband, moderate roll-off
% - Chebyshev I: faster roll-off for same order but passband ripple (may
slightly
%   distort amplitude of ECG waves)
% - Chebyshev II: faster roll-off and controlled stopband attenuation with
%   stopband ripple (passband is monotonic like Butterworth)
%
% Recommendation printed:
fprintf('=== Practical Recommendation ===\n');
fprintf('For this ECG example (Fc = %g Hz):\n', Fc);
fprintf('- Butterworth gives a flat passband (no ripple), preserving
waveform\n');
fprintf('  shapes well, but has a gentler transition (may leave some 50 Hz
energy).\n');
fprintf('- Chebyshev I gives a steeper transition but introduces passband
ripple\n');
fprintf('  (may slightly alter ECG amplitude/peaks). Use if you must
maximize roll-off\n');
fprintf('  and can tolerate small passband amplitude variations.\n');
fprintf('- Chebyshev II gives a steep transition with monotonic passband and
strong\n');
fprintf('  stopband attenuation (good compromise when you want to suppress
50 Hz).\n\n');

% Short verdict:
% Prefer Chebyshev Type II (with adequate stopband attenuation) if the
primary
% goal is to strongly attenuate 50 Hz and higher while keeping a monotonic
% passband. If absolute flatness in the passband is required (no amplitude
% ripple at all), choose Butterworth and increase order if needed.
fprintf('Verdict: For typical ECG denoising where strong 50 Hz suppression
is\n');
fprintf('needed without passband ripples, Chebyshev Type II (rs = %g dB) is
a good\n', rs);
fprintf('choice here. If minimal passband distortion is paramount, use
Butterworth\n');
fprintf('and consider increasing filter order to get more attenuation.\n\n');

% End of script
```